Physical AI応用1 最終課題レポート

1. 課題の概要

Genesisシミュレータ上のFranka Emika
Pandaロボットアームに対し、「青いブロックを取って」
のような自然言語の指示文を与えると、対象のブロックを認識して掴み、別の場所へ運ぶ一連の動作を実現した。AIの活用として、(1) 指示文の解釈に大規模言語モデル(LLM)による推論、 (2) カメラ画像からの物体位置推定、の2点を実装した。

2. 目的

- ロボットシミュレータ(Genesis)の基本的な操作(逆運動学、力制御、カメラ録画)を習得する。
- 自然言語指示をロボットの行動に変換する、LLMを用いたタスクプランニングを実装する。
- シミュレータの特権的な正解情報(物体の真の座標)に頼らず、カメラ画像のみから物体位置を

推定する知覚パイプラインを構築し、その精度を定量評価する。

3. システム構成

3段階で構築した。

Stage1: 固定キューブのピック&リフト（土台）

- 地面・4cmキューブ1個・Frankaを配置。
- `inverse_kinematics`で目標関節角を求め、現在角から関節空間で線形補間する独自の

`move_to`関数で動作させる(OMPL依存の`plan_path`は使わない)。

- 指は位置制御で閉じるとキューブを弾いてしまうため、力制御(`control_dofs_force`)で

一定の把持力を与える。

Stage2: 自然言語指示によるキューブ選択

- 赤・青・緑のキューブ3個をランダムな位置に配置。
- 各キューブの色と座標の一覧、およびユーザーの指示文をLLMに渡し、

`{"target_color": "...", "reason": "..."}`形式のJSONのみで対象キューブを回答させる。

- LLMバックエンドは環境変数で切り替え可能にした。既定はローカルで動作する

Ollama(`qwen2.5:3b-instruct`)で、APIキーやクレジットを使わずに動作する。 Anthropic APIへの切り替えにも対応している。

- 複数の指示を連続実行するデモモードを実装し、3キューブをそれぞれ異なる場所へ

運ぶ様子を確認した。

Stage3: カメラ画像からの位置推定

- シミュレータの正解座標(`entity.get_pos()`)を使わず、机を俯瞰する固定カメラの

RGB-D画像のみからキューブの3次元位置を推定するパイプラインを実装。

- RGB画像上で各キューブの色(しきい値近傍)に該当する画素のマスクを取り、その重心

ピクセル座標を対象物の画像上位置とする。

- マスク内の深度値の中央値を取り、カメラの内部パラメータ(`intrinsics`)・外部

パラメータ(`extrinsics`)を用いて画素+深度からワールド座標系の3次元点へ逆投影する。

- 推定した座標のみをLLMとピック&プレース動作に渡し、シミュレータの正解座標には

一切アクセスしない構成にした。

- 推定誤差は`outputs/stage3_perception_errors.csv`に記録し、定量的に評価した。

4. 工夫した点

1 `plan_path`**を避けた独自モーションプリミティブ**: GenesisのサンプルはOMPLベースの

`plan_path`を使う例が多いが、依存解決で環境構築が不安定になりやすいため、
逆運動学の結果へ関節空間で単純に線形補間するだけの`move_to`を実装した。実用上
十分な滑らかさと速度で動作することを確認した。

1 **力制御によるロバストな把持**: 指を位置制御で閉じると、接触時にキューブへ

過大な反力が生じて弾いてしまう現象が見られた。力制御(`control_dofs_force`)に
切り替えることで、キューブを潰さずに安定して把持できるようにした。また、
持ち上げ・移動中も毎ステップ同じ力指令を送り続けないと把持が緩むことが
分かったため、移動用の`move_to`に`grip_force`引数を追加し、移動中も把持力を
維持できるようにした。

1 **LLMバックエンドの抽象化**: 当初Anthropic APIを利用する設計だったが、開発中に

APIクレジットを使い切ってしまい、提出期限までに補充できない事態が発生した。
そこで`llm_planner.py`のLLM呼び出し部分を抽象化し、環境変数`LLM_BACKEND`で
Ollama(ローカル・無料)とAnthropic API(クラウド)を切り替えられるようにした。
これにより、APIキーやクレジットがなくても「AIによる推論」の要件を満たしつつ、
将来クラウドAPIに戻すことも容易な設計にできた。

1 **JSON出力の頑健なパース**: LLMの出力は厳密にJSONのみに従うとは限らないため、

正規表現で最初の`{...}`ブロックを抽出し、パース失敗やあり得ない色が返った場合は
エラー内容をプロンプトに含めて再試行するリトライ機構を実装した。

1 **深度画像の系統誤差への対処**: Stage3で、キューブ中心座標を深度から直接推定すると

常に約2cm高めに推定される系統誤差が生じた。これは俯瞰カメラが捉える深度が
キューブの「上面」であり「中心」ではないためと判明し、真下視点という条件下で

キューブ高さの半分だけ補正することで、推定誤差を約2cmから1〜4mm程度まで 改善した。

5. うまくいかなかった点・制約と考察

- **非ASCIIパスでのGenesis実行不可**: プロジェクトフォルダ名に日本語を含む環境で

venvを作成すると、Genesisの内部コンパイラ(LLVM)がランタイムのbitcodeファイルを見つけられずクラッシュする問題に遭遇した。venv自体をASCIIパス(`C:\dev\venvs\genesis_env`)に作成することで回避したが、根本的にはWindows上のGenesisパッケージの非ASCIIパス対応が不十分であることが原因と考えられる。

- **APIクレジット不足によるAnthropic API未検証**: 当初予定していたAnthropic API

(Claude)での実運用検証ができず、ローカルLLM(Ollama)での検証に切り替えた。軽量モデル(3Bパラメータ)でも、JSON形式の指定とformat="json"オプションの併用により、今回のような単純な色選択タスクでは安定して正しい判断ができることを確認できた。より複雑な推論を要するタスクでは、モデルサイズと応答品質のトレードオフが課題になると考えられる。

- **色セグメンテーションの限界**: 現在の物体検出は色のしきい値処理のみに依存して

おり、照明条件の変化や、同系色の物体が複数ある場合の識別には対応できない。当初はClaudeのビジョン入力を使ったピクセル位置推定との比較も計画していたが、時間の制約により実装を見送った。将来的な拡張として、VLM(Vision-Language Model)による検出と比較することで、頑健性の違いを定量評価したい。

- **把持の成功判定の単純さ**: 現状はキューブの最終位置と目標位置のユークリッド距離

のみで成功を判定しており、途中で把持が外れて別の経路で目標位置近くに転がった場合などを誤って成功とみなす可能性がある。より厳密には、持ち上げ区間でのグリッパー・物体間の接触力や相対速度を監視するべきである。

6. まとめ

固定キューブのピック&リフトから出発し、自然言語指示によるLLMベースのタスクプランニング、さらにシミュレータの正解情報に頼らないカメラ画像ベースの知覚へと段階的に拡張することで、「言葉で指示されたブロックを掴むロボットアーム」を実現した。特に、開発途中のAPIクレジット制約に対してLLMバックエンドを抽象化し、ローカルLLMで代替する設計変更を行ったことは、実運用を見据えた柔軟性の面で工夫点として挙げられる。